

DEFINITION OF NP-COMPLETENESS

DEFINITION 7.34

A language B is *NP-complete* if it satisfies two conditions:

1. B is in NP, and
2. every A in NP is polynomial time reducible to B .

THEOREM 7.35

If B is NP-complete and $B \in P$, then $P = NP$.

PROOF This theorem follows directly from the definition of polynomial time reducibility.

THEOREM 7.36

If B is NP-complete and $B \leq_P C$ for C in NP, then C is NP-complete.

PROOF We already know that C is in NP, so we must show that every A in NP is polynomial time reducible to C . Because B is NP-complete, every language in NP is polynomial time reducible to B , and B in turn is polynomial time reducible to C . Polynomial time reductions compose; that is, if A is polynomial time reducible to B and B is polynomial time reducible to C , then A is polynomial time reducible to C . Hence every language in NP is polynomial time reducible to C .

THE COOK–LEVIN THEOREM

Once we have one NP-complete problem, we may obtain others by polynomial time reduction from it. However, establishing the first NP-complete problem is more difficult. Now we do so by proving that *SAT* is NP-complete.

THEOREM 7.37

SAT is NP-complete.²

This theorem implies Theorem 7.27.

²An alternative proof of this theorem appears in Section 9.3.

PROOF IDEA Showing that *SAT* is in NP is easy, and we do so shortly. The hard part of the proof is showing that any language in NP is polynomial time reducible to *SAT*.

To do so, we construct a polynomial time reduction for each language A in NP to *SAT*. The reduction for A takes a string w and produces a Boolean formula ϕ that simulates the NP machine for A on input w . If the machine accepts, ϕ has a satisfying assignment that corresponds to the accepting computation. If the machine doesn't accept, no assignment satisfies ϕ . Therefore, w is in A if and only if ϕ is satisfiable.

Actually constructing the reduction to work in this way is a conceptually simple task, though we must cope with many details. A Boolean formula may contain the Boolean operations AND, OR, and NOT, and these operations form the basis for the circuitry used in electronic computers. Hence the fact that we can design a Boolean formula to simulate a Turing machine isn't surprising. The details are in the implementation of this idea.

PROOF First, we show that *SAT* is in NP. A nondeterministic polynomial time machine can guess an assignment to a given formula ϕ and accept if the assignment satisfies ϕ .

Next, we take any language A in NP and show that A is polynomial time reducible to *SAT*. Let N be a nondeterministic Turing machine that decides A in n^k time for some constant k . (For convenience, we actually assume that N runs in time $n^k - 3$; but only those readers interested in details should worry about this minor point.) The following notion helps to describe the reduction.

A **tableau** for N on w is an $n^k \times n^k$ table whose rows are the configurations of a branch of the computation of N on input w , as shown in the following figure.

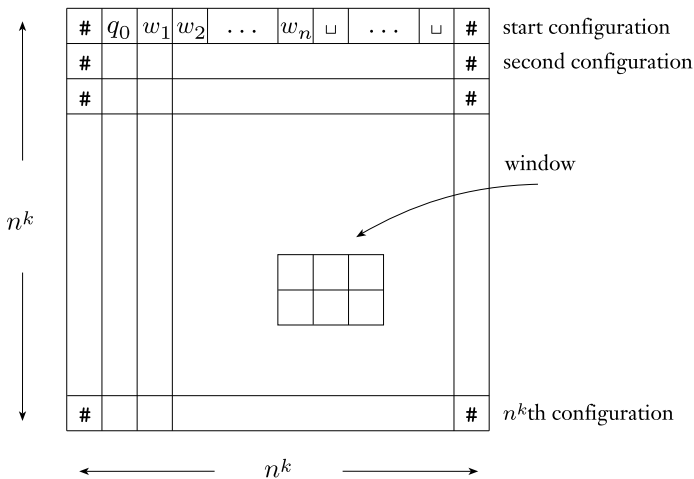


FIGURE 7.38
A tableau is an $n^k \times n^k$ table of configurations

For convenience later, we assume that each configuration starts and ends with a # symbol. Therefore, the first and last columns of a tableau are all #s. The first row of the tableau is the starting configuration of N on w , and each row follows the previous one according to N 's transition function. A tableau is *accepting* if any row of the tableau is an accepting configuration.

Every accepting tableau for N on w corresponds to an accepting computation branch of N on w . Thus, the problem of determining whether N accepts w is equivalent to the problem of determining whether an accepting tableau for N on w exists.

Now we get to the description of the polynomial time reduction f from A to SAT . On input w , the reduction produces a formula ϕ . We begin by describing the variables of ϕ . Say that Q and Γ are the state set and tape alphabet of N , respectively. Let $C = Q \cup \Gamma \cup \{\#\}$. For each i and j between 1 and n^k and for each s in C , we have a variable, $x_{i,j,s}$.

Each of the $(n^k)^2$ entries of a tableau is called a *cell*. The cell in row i and column j is called $cell[i, j]$ and contains a symbol from C . We represent the contents of the cells with the variables of ϕ . If $x_{i,j,s}$ takes on the value 1, it means that $cell[i, j]$ contains an s .

Now we design ϕ so that a satisfying assignment to the variables does correspond to an accepting tableau for N on w . The formula ϕ is the AND of four parts: $\phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$. We describe each part in turn.

As we mentioned previously, turning variable $x_{i,j,s}$ on corresponds to placing symbol s in $cell[i, j]$. The first thing we must guarantee in order to obtain a correspondence between an assignment and a tableau is that the assignment turns on exactly one variable for each cell. Formula ϕ_{cell} ensures this requirement by expressing it in terms of Boolean operations:

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right].$$

The symbols $\bigwedge$ and $\bigvee$ stand for iterated AND and OR. For example, the expression in the preceding formula

$$\bigvee_{s \in C} x_{i,j,s}$$

is shorthand for

$$x_{i,j,s_1} \vee x_{i,j,s_2} \vee \cdots \vee x_{i,j,s_l}$$

where $C = \{s_1, s_2, \dots, s_l\}$. Hence ϕ_{cell} is actually a large expression that contains a fragment for each cell in the tableau because i and j range from 1 to n^k . The first part of each fragment says that at least one variable is turned on in the corresponding cell. The second part of each fragment says that no more than one variable is turned on (literally, it says that in each pair of variables, at least one is turned off) in the corresponding cell. These fragments are connected by $\wedge$ operations.

The first part of ϕ_{cell} inside the brackets stipulates that at least one variable that is associated with each cell is on, whereas the second part stipulates that no more than one variable is on for each cell. Any assignment to the variables that satisfies ϕ (and therefore ϕ_{cell}) must have exactly one variable on for every cell. Thus, any satisfying assignment specifies one symbol in each cell of the table. Parts ϕ_{start} , ϕ_{move} , and ϕ_{accept} ensure that these symbols actually correspond to an accepting tableau as follows.

Formula ϕ_{start} ensures that the first row of the table is the starting configuration of N on w by explicitly stipulating that the corresponding variables are on:

$$\begin{aligned}\phi_{\text{start}} = & x_{1,1,\#} \wedge x_{1,2,q_0} \wedge \\ & x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \dots \wedge x_{1,n+2,w_n} \wedge \\ & x_{1,n+3,\sqcup} \wedge \dots \wedge x_{1,n^k-1,\sqcup} \wedge x_{1,n^k,\#}.\end{aligned}$$

Formula ϕ_{accept} guarantees that an accepting configuration occurs in the tableau. It ensures that q_{accept} , the symbol for the accept state, appears in one of the cells of the tableau by stipulating that one of the corresponding variables is on:

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}.$$

Finally, formula ϕ_{move} guarantees that each row of the tableau corresponds to a configuration that legally follows the preceding row's configuration according to N 's rules. It does so by ensuring that each 2×3 window of cells is legal. We say that a 2×3 window is **legal** if that window does not violate the actions specified by N 's transition function. In other words, a window is legal if it might appear when one configuration correctly follows another.³

For example, say that a , b , and c are members of the tape alphabet, and q_1 and q_2 are states of N . Assume that when in state q_1 with the head reading an a , N writes a b , stays in state q_1 , and moves right; and that when in state q_1 with the head reading a b , N nondeterministically either

1. writes a c , enters q_2 , and moves to the left, or
2. writes an a , enters q_2 , and moves to the right.

Expressed formally, $\delta(q_1, a) = \{(q_1, b, R)\}$ and $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$. Examples of legal windows for this machine are shown in Figure 7.39.

³We could give a precise definition of **legal window** here, in terms of the transition function. But doing so is quite tedious and would distract us from the main thrust of the proof argument. Anyone desiring more precision should refer to the related analysis in the proof of Theorem 5.15, the undecidability of the Post Correspondence Problem.

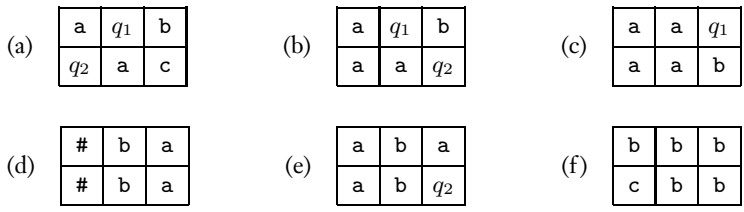


FIGURE 7.39
Examples of legal windows

In Figure 7.39, windows (a) and (b) are legal because the transition function allows N to move in the indicated way. Window (c) is legal because, with q_1 appearing on the right side of the top row, we don't know what symbol the head is over. That symbol could be an a , and q_1 might change it to a b and move to the right. That possibility would give rise to this window, so it doesn't violate N 's rules. Window (d) is obviously legal because the top and bottom are identical, which would occur if the head weren't adjacent to the location of the window. Note that $\#$ may appear on the left or right of both the top and bottom rows in a legal window. Window (e) is legal because state q_1 reading a b might have been immediately to the right of the top row, and it would then have moved to the left in state q_2 to appear on the right-hand end of the bottom row. Finally, window (f) is legal because state q_1 might have been immediately to the left of the top row, and it might have changed the b to a c and moved to the left.

The windows shown in the following figure aren't legal for machine N .

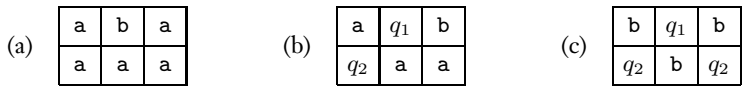


FIGURE 7.40
Examples of illegal windows

In window (a), the central symbol in the top row can't change because a state wasn't adjacent to it. Window (b) isn't legal because the transition function specifies that the b gets changed to a c but not to an a . Window (c) isn't legal because two states appear in the bottom row.

CLAIM 7.41

If the top row of the tableau is the start configuration and every window in the tableau is legal, each row of the tableau is a configuration that legally follows the preceding one.

We prove this claim by considering any two adjacent configurations in the tableau, called the upper configuration and the lower configuration. In the upper configuration, every cell that contains a tape symbol and isn't adjacent to a state symbol is the center top cell in a window whose top row contains no states. Therefore, that symbol must appear unchanged in the center bottom of the window. Hence it appears in the same position in the bottom configuration.

The window containing the state symbol in the center top cell guarantees that the corresponding three positions are updated consistently with the transition function. Therefore, if the upper configuration is a legal configuration, so is the lower configuration, and the lower one follows the upper one according to N 's rules. Note that this proof, though straightforward, depends crucially on our choice of a 2×3 window size, as Problem 7.41 shows.

Now we return to the construction of ϕ_{move} . It stipulates that all the windows in the tableau are legal. Each window contains six cells, which may be set in a fixed number of ways to yield a legal window. Formula ϕ_{move} says that the settings of those six cells must be one of these ways, or

$$\phi_{\text{move}} = \bigwedge_{1 \leq i < n^k, 1 \leq j < n^k} (\text{the } (i, j)\text{-window is legal}).$$

The (i, j) -window has $\text{cell}[i, j]$ as the upper central position. We replace the text “the (i, j) -window is legal” in this formula with the following formula. We write the contents of six cells of a window as $a_1, \dots, a_6$.

$$\bigvee_{\substack{a_1, \dots, a_6 \\ \text{is a legal window}}} (x_{i, j-1, a_1} \wedge x_{i, j, a_2} \wedge x_{i, j+1, a_3} \wedge x_{i+1, j-1, a_4} \wedge x_{i+1, j, a_5} \wedge x_{i+1, j+1, a_6})$$

Next, we analyze the complexity of the reduction to show that it operates in polynomial time. To do so, we examine the size of ϕ . First, we estimate the number of variables it has. Recall that the tableau is an $n^k \times n^k$ table, so it contains n^{2k} cells. Each cell has l variables associated with it, where l is the number of symbols in C . Because l depends only on the TM N and not on the length of the input n , the total number of variables is $O(n^{2k})$.

We estimate the size of each of the parts of ϕ . Formula ϕ_{cell} contains a fixed-size fragment of the formula for each cell of the tableau, so its size is $O(n^{2k})$. Formula ϕ_{start} has a fragment for each cell in the top row, so its size is $O(n^k)$. Formulas ϕ_{move} and ϕ_{accept} each contain a fixed-size fragment of the formula for each cell of the tableau, so their size is $O(n^{2k})$. Thus, ϕ 's total size is $O(n^{2k})$. That bound is sufficient for our purposes because it shows that the size of ϕ is polynomial in n . If it were more than polynomial, the reduction wouldn't have any chance of generating it in polynomial time. (Actually, our estimates are low by a factor of $O(\log n)$ because each variable has indices that can range up to n^k and so may require $O(\log n)$ symbols to write into the formula, but this additional factor doesn't change the polynomiality of the result.)

To see that we can generate the formula in polynomial time, observe its highly repetitive nature. Each component of the formula is composed of many nearly

identical fragments, which differ only at the indices in a simple way. Therefore, we may easily construct a reduction that produces ϕ in polynomial time from the input w .

Thus, we have concluded the proof of the Cook–Levin theorem, showing that *SAT* is NP-complete. Showing the NP-completeness of other languages generally doesn’t require such a lengthy proof. Instead, NP-completeness can be proved with a polynomial time reduction from a language that is already known to be NP-complete. We can use *SAT* for this purpose; but using *3SAT*, the special case of *SAT* that we defined on page 302, is usually easier. Recall that the formulas in *3SAT* are in conjunctive normal form (cnf) with three literals per clause. First, we must show that *3SAT* itself is NP-complete. We prove this assertion as a corollary to Theorem 7.37.

COROLLARY 7.42

3SAT is NP-complete.

PROOF Obviously *3SAT* is in NP, so we only need to prove that all languages in NP reduce to *3SAT* in polynomial time. One way to do so is by showing that *SAT* polynomial time reduces to *3SAT*. Instead, we modify the proof of Theorem 7.37 so that it directly produces a formula in conjunctive normal form with three literals per clause.

Theorem 7.37 produces a formula that is already almost in conjunctive normal form. Formula ϕ_{cell} is a big AND of subformulas, each of which contains a big OR and a big AND of ORs. Thus, ϕ_{cell} is an AND of clauses and so is already in cnf. Formula ϕ_{start} is a big AND of variables. Taking each of these variables to be a clause of size 1, we see that ϕ_{start} is in cnf. Formula ϕ_{accept} is a big OR of variables and is thus a single clause. Formula ϕ_{move} is the only one that isn’t already in cnf, but we may easily convert it into a formula that is in cnf as follows.

Recall that ϕ_{move} is a big AND of subformulas, each of which is an OR of ANDs that describes all possible legal windows. The distributive laws, as described in Chapter 0, state that we can replace an OR of ANDs with an equivalent AND of ORs. Doing so may significantly increase the size of each subformula, but it can only increase the total size of ϕ_{move} by a constant factor because the size of each subformula depends only on N . The result is a formula that is in conjunctive normal form.

Now that we have written the formula in cnf, we convert it to one with three literals per clause. In each clause that currently has one or two literals, we replicate one of the literals until the total number is three. In each clause that has more than three literals, we split it into several clauses and add additional variables to preserve the satisfiability or unsatisfiability of the original.

For example, we replace clause $(a_1 \vee a_2 \vee a_3 \vee a_4)$, wherein each a_i is a literal, with the two-clause expression $(a_1 \vee a_2 \vee z) \wedge (\bar{z} \vee a_3 \vee a_4)$, wherein z is a new

$$(a_1 \vee a_2 \vee \cdots \vee a_l),$$
$$(a_1 \vee a_2 \vee z_1) \wedge (\overline{z_1} \vee a_3 \vee z_2) \wedge (\overline{z_2} \vee a_4 \vee z_3) \wedge \cdots \wedge (\overline{z_{l-3}} \vee a_{l-1} \vee a_l).$$

7.5

The phenomenon of NP-completeness is widespread. NP-complete problems appear in many fields. For reasons that are not well understood, most naturally occurring NP-problems are known either to be in P or to be NP-complete. If you seek a polynomial time algorithm for a new NP-problem, spending part of your effort attempting to prove it NP-complete is sensible because doing so may prevent you from working to find a polynomial time algorithm that doesn't exist.

In this section, we present additional theorems showing that various languages are NP-complete. These theorems provide examples of the techniques that are used in proofs of this kind. Our general strategy is to exhibit a polynomial time reduction from $3SAT$ to the language in question, though we sometimes reduce from other NP-complete languages when that is more convenient.

COROLLARY 7.43

Copyright 2012 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.